

Introduzione a Python

Variabili in Python

- Non c'è dichiarazione del tipo, il tipo si assegna dinamicamente in base al valore.
- Puoi riassegnare variabili con valori di tipo diverso senza problemi.
- Convenzione: nomi in **snake_case** (es: `nome_variabile`).
- Python è case-sensitive.

```
1 x = 42          # x      un int
2 x = "ciao"      # ora x   una stringa
```

- Variabili globali e locali seguono scope simili ad altri linguaggi, ma attenzione alle variabili globali in funzioni (usa `global` o `nonlocal` per modificarle dentro funzioni).

Tipi di dati

Numerici

- **int**: interi illimitati (non overflow come in C/C++).
- **float**: numeri in virgola mobile, doppia precisione.
- **complex**: numeri complessi, es: `3 + 4j`.

```
1 a = 10          # int
2 b = 3.14         # float
3 c = 2 + 3j       # complex
```

Stringhe (str)

Immutabili, puoi indicarle con `'...'`, `"..."`, `'''...'''` o `"""..."""`.

Supportano escape, formattazione e metodi ricchi.

```
1 s = "Hello\nWorld"
2 print(len(s))    # lunghezza
3 print(s.upper()) # maiuscolo
4 print(f"Ciao {nome}") # f-string per interpolazione (Python
  3.6+)
```

Liste

- Struttura mutabile, indicizzabile, eterogenea.
- Puoi aggiungere, rimuovere, ordinare, etc.

```
1 lista = [1, 2, "tre", 4.0]
2 lista.append(5)
3 print(lista[2]) # "tre"
```

Tuple

- Liste immutabili, usate per dati che non devono cambiare.
- Possono essere usate come chiavi in dizionari (liste no).

```
1 t = (1, 2, 3)
```

Dizionari (dict)

- Collezione key-value, chiavi univoche.
- Le chiavi possono essere tipi immutabili (stringhe, tuple, numeri).

```
1 diz = {"nome": "Giacomo", "eta": 30}
2 print(diz["nome"]) # Giacomo
3 diz["eta"] = 31    # aggiorna valore
```

Set

- Collezione non ordinata di elementi unici.
- Operazioni matematiche (unione, intersezione).

```
1 s = {1, 2, 3, 3}
2 print(s)  # {1, 2, 3}
```

Cicli

For

- Itera su ogni elemento di una sequenza o iterable (`range`, lista, stringa, dizionario, file, generatori...).

```
1 for i in range(3):
2     print(i)  # 0,1,2
3
4 for c in "ciao":
5     print(c)
```

- Puoi iterare su chiavi o valori di un dizionario:

```
1 for k, v in diz.items():
2     print(k, v)
```

While

- Simile a C/Java, esegue finché la condizione è vera.

```
1 count = 0
2 while count < 3:
3     print(count)
4     count += 1
```

- Usa `break` per uscire dal ciclo.
- Usa `continue` per saltare un'iterazione.

Funzioni

- Definite con `def nome_funzione(parametri):`.
- Indentazione obbligatoria (4 spazi o tab).
- Non serve dichiarare tipi (ma puoi usare type hints opzionali).
- Funzioni possono ritornare valori con `return` (di default ritornano `None`).
- Puoi ritornare tuple per più valori.

```
1 def moltiplica(a: int, b: int) -> int:
2     return a * b
3
4 def min_max(lista: list) -> tuple:
5     return min(lista), max(lista)
```

- Funzioni possono avere parametri con valori di default:

```
1 def saluto(nome="Mondo"):
2     print(f"Ciao {nome}!")
```

- Puoi usare parametri variabili con `*args` (posizionali) e `**kwargs` (key-word args):

```
1 def somma(*numeri):
2     return sum(numeri)
3
4 def stampa_info(**info):
5     for k, v in info.items():
6         print(f"{k}: {v}")
```

Fondamenti OOP in Python

- Sintassi più concisa rispetto a Java o PHP.
- Ogni metodo (funzione dentro classe) deve avere il parametro `self` (riferimento all'istanza).

- Non serve scrivere tipi di accesso (public/private/protected), ma per convenzione:
- `_attributo` → “protetto” (non usare fuori dalla classe).
- `__attributo` → “privato” (name mangling, evita collisioni).
- Costruttore: `__init__(self, ...)`.
- Ereditarietà: `class Figlio(Genitore):`.
- Per chiamare il costruttore o altri metodi del genitore: `super()`.

Esempio completo

```

1 class Animale:
2     def __init__(self, nome):
3         self.nome = nome
4
5     def parla(self):
6         print(f"{self.nome} fa un verso")
7
8 class Cane(Animale):
9     def __init__(self, nome, razza):
10        super().__init__(nome)
11        self.razza = razza
12
13    def parla(self):
14        print(f"{self.nome} abbaia")
15
16    def info(self):
17        print(f"Cane di razza {self.razza}")
18
19 # Uso
20 fido = Cane("Fido", "Labrador")
21 fido.parla()    # Fido abbaia
22 fido.info()     # Cane di razza Labrador

```

Altri dettagli OOP

- Proprietà con decoratori `@property` per getter/setter.

```

1 class Rettangolo:
2     def __init__(self, base, altezza):
3         self._base = base
4         self._altezza = altezza
5
6     @property
7     def area(self):
8         return self._base * self._altezza
9
10 r = Rettangolo(4, 5)
11 print(r.area) # 20

```

- Metodi statici e di classe:

```

1 class Matematica:
2     @staticmethod
3     def somma(a, b):
4         return a + b
5
6     @classmethod
7     def crea_da_lista(cls, lista):
8         return cls(*lista)
9
10 print(Matematica.somma(2, 3)) # 5

```